快速排序

本章内容

- 快速排序的描述
- 快速排序的性能
- 快速排序的随机化版本
- 快速排序分析

7.1快速排序的描述

- 快速排序是一种排序算法，对包含n个数的输入数组，最坏情况运行时间为$\Theta(n^2)$ 。
- 虽然最坏情况运行时间比较差，但快速排序通常是用于排序的最佳的使用选择，这是因为其平均性能相当好：期望的运行时间为$\Theta(nlgn)$，且$\Theta(nlgn)$记号中隐含的常数因子很小。
- 快速排序还能就地排序。

快速排序的步骤

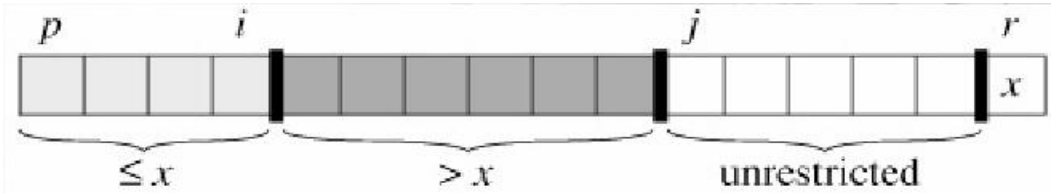
- 快速排序分三个步骤
  - 分解：数组A[p..r]被划分成两个子数组A[p..q-1]和A[q+1..r]，使得A[p..q-1]中的每个元素都小于等于A(q)，而且，小于A[q+1..r]中的元素。下标q也在这个划分过程中进行计算。
  - 解决：通过递归调用快速排序，对子数组A[p..q-1]和A[q+1..r]排序。
  - 合并：因为两个子数组是就地排序的，将它们合并不需要操作，整个数组A[p..r]已排序

快速排序程序

- QUICKSORT(A,p,r)

```
1 if p < r
2   then q ←PARTITION(A,p,r)
3        QUICKSORT(A,p,q-1)
4        QUICKSORT(A,q+1,r)
```

数组划分程序

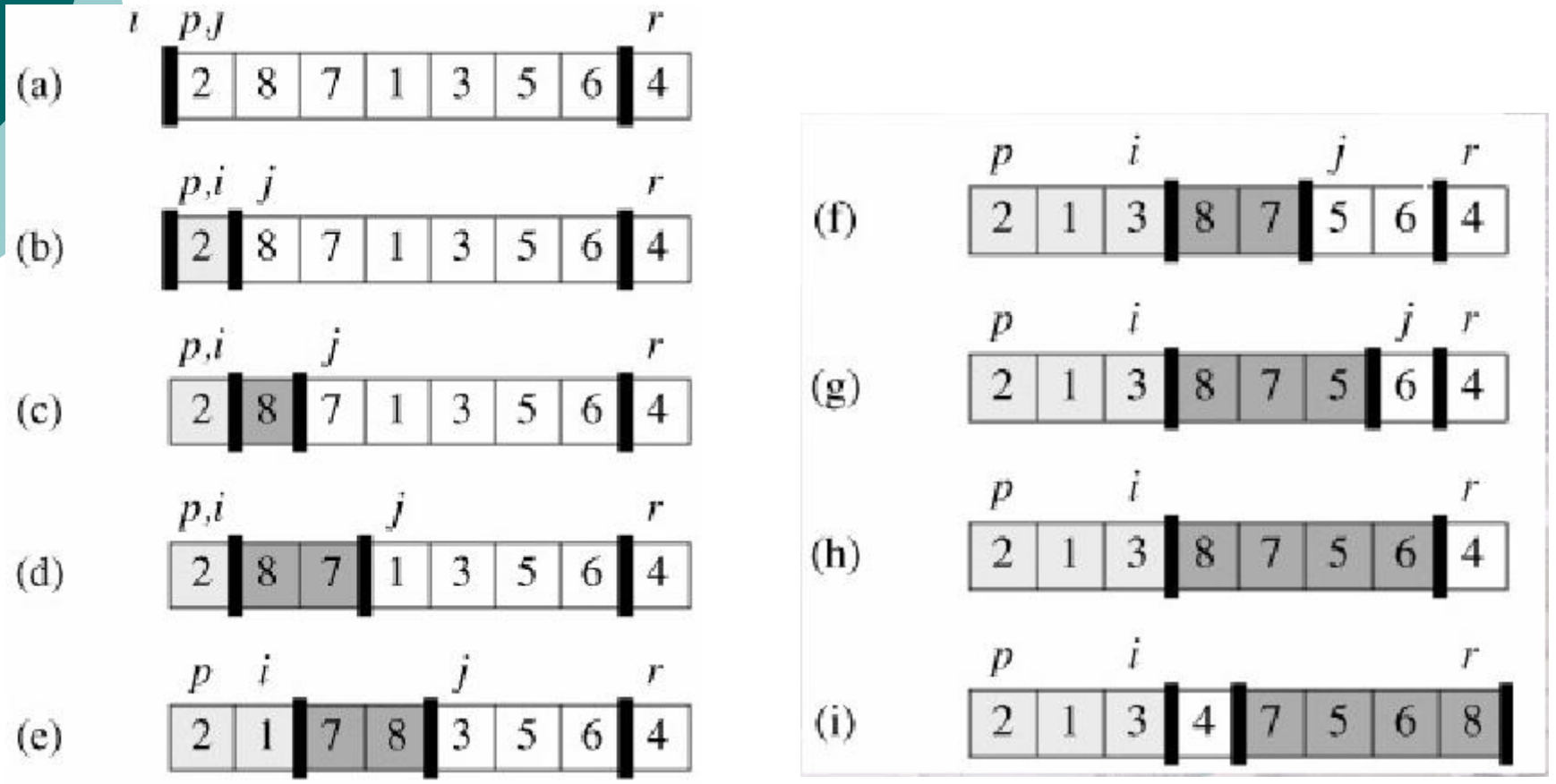


- PARTITION(A,p,r)

```
1 x←A[r]
2 i←p-1
3 for j←p to r-1
4         do if A[j]≤x
5                   then i ←i+1
6                             exchange A[i]←→A[j]
7 exchange A[i+1]←→A[r]
8 return i+1
```

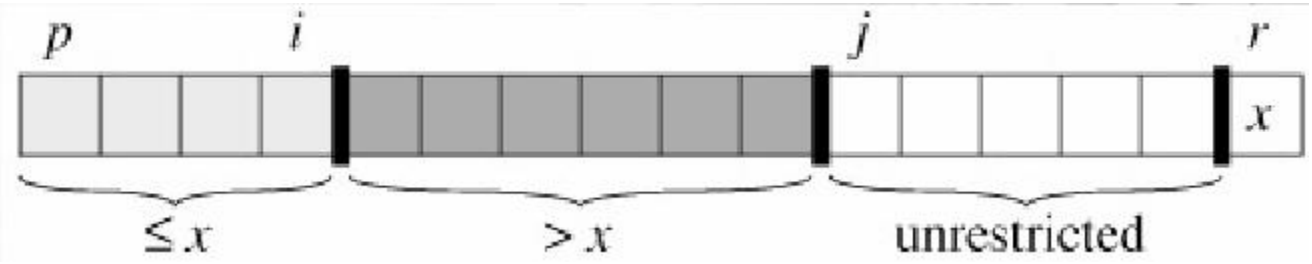
快速排序图示

下图是8元素数组上运行PARTITION过程的示例

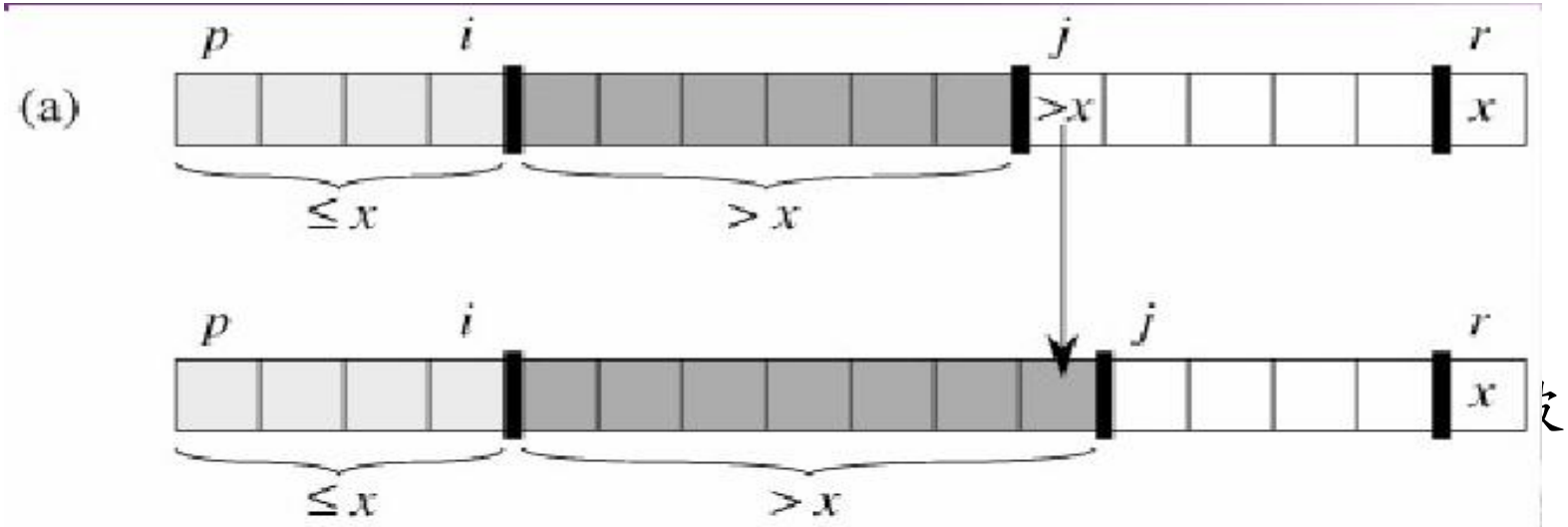


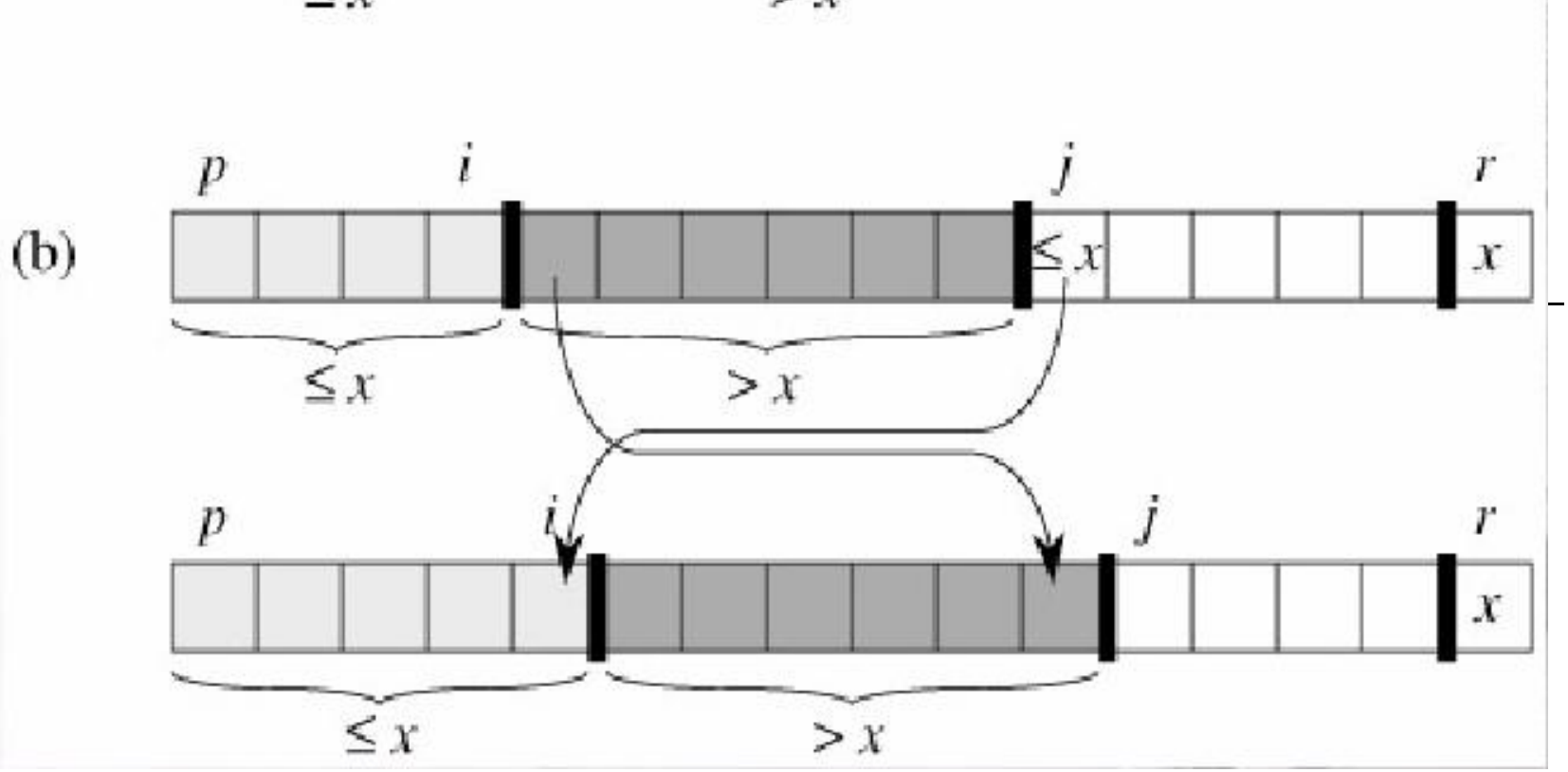
循环不变式

- 在第3-6行的每次循环之前，对于下标k
  - 若$p \le k \le i$, 则$A[k] \le x$.
  - 若$i + 1 \le k \le j - 1$, 则 $A[k] > x$.
  - 若$k = r$, 则 $A[k] = x$.
- 下图说明了这种情况。j与r-1之间的下标并未被包含进这三种情况，它们的元素和x也没有明确的关系



- 如果下图所示，要考虑两种情况，具体取决于第4行中测试的结果。
  - 图a显示当A[j]>x时所做的处理；循环中的唯一操作是j增加1。
    - 在j增加1后，条件2对A[j-1]成立，且所有其他项保持不变。





- 图b显示当A[j]<=x时所做的处理：将i增加1，交互A[i]和A[j]，再将j增加1。
  - 因为进行了交换，现在有A[i] <=x，因而条件1满足。
  - 类似地，还有A[j-1]>x，因为根据循环不变式，被交换进A[j-1]的项目是大于x的

PARTITION运行时间

- PARTITION在子数组$A[p..r]$上的执行时间为$\Theta(n)$, $n = r - p + 1$。

7.2 快速排序的性能

- 快速排序的运行时间与划分是否对称有关，我们分别就以下三种情况进行讨论
  - 最坏情况划分
  - 最佳情况划分
  - 平衡的划分

最坏情况划分

- 快速排序的最坏情况划分行为发生在划分过程产生的两个区域分别包含n-1个元素和1个元素的时候。
- 假设算法的每一次递归调用中都出现了这种不对称划分。划分的时间代价为$\Theta(n)$。
- 因为对一个大小为0的数组进行递归调用后，返回$T(0)=\Theta(1)$,故算法的运行时间可以递归的表示为:

$$T(n)=T(n-1)+T(0)+\Theta(n)=T(n-1)+\Theta(n)$$

- 用代换法，可以证明$T(n)=\Theta(n^2)$(后面会证明)

最佳情况划分

- 在PARTITION可能做的最平衡的划分中，得到的两个子问题的大小都不大可能大于n/2。
- 在这种情况下，快速排序运行的速度要快的多。这时，表达其运行时间的递归式为

$$T(n)\leq 2T(n/2)+\Theta(n)$$

根据主定理，该递归式的解为

$$T(n)=O(n\lg n)。$$

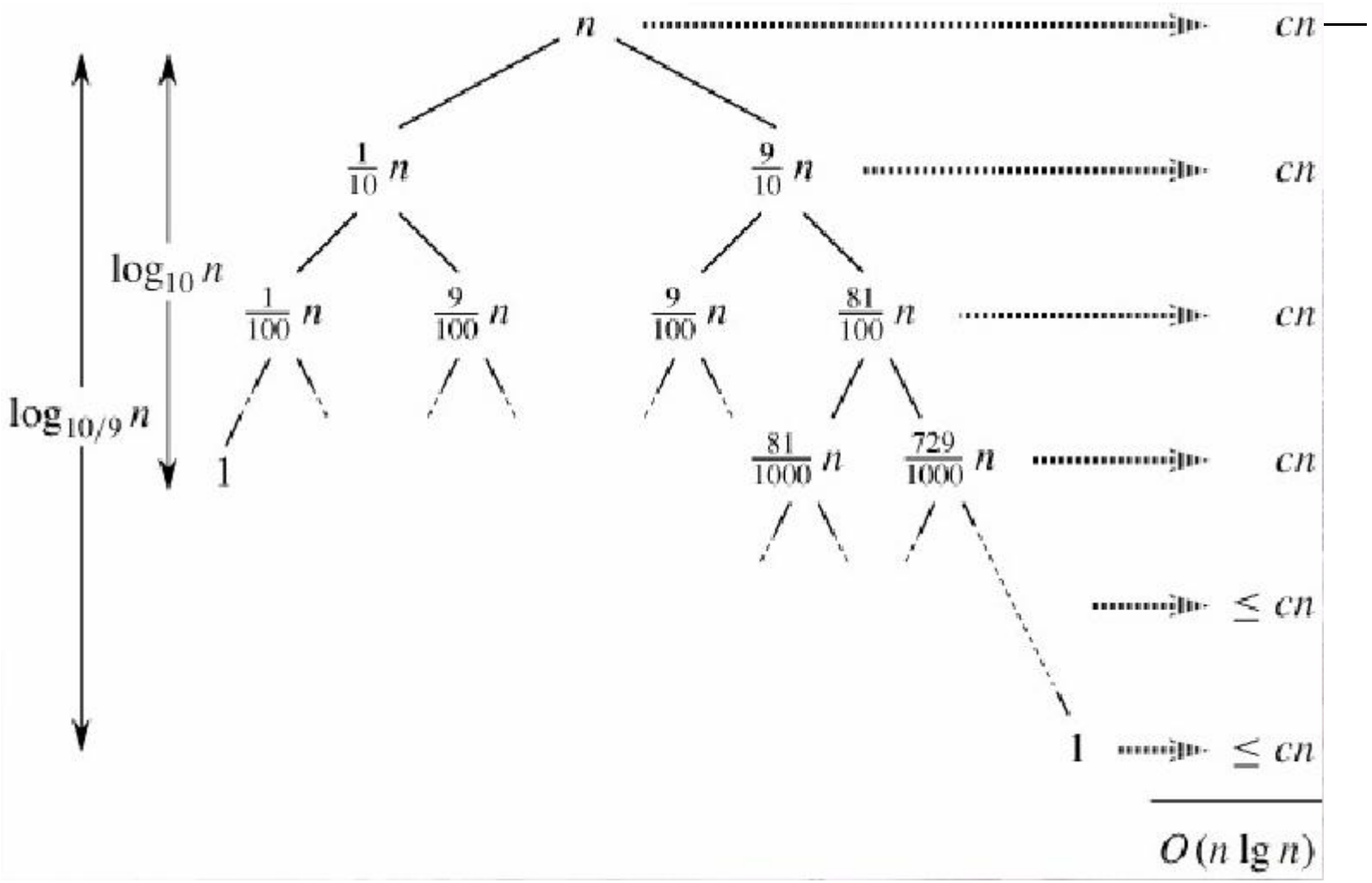
平衡的划分

- 假设划分过程总是产生9：1的划分，乍一看这种划分很不平衡，这时快速排序的递归式为

$$T(n) \le T(9n/10) + T(n/10) + cn$$

这个递归式对应的递归树如下图所示：

平衡的划分


n
cn
1/10 n
9/10 n
cn
log10 n
1/100 n
9/100 n
9/100 n
81/100 n
cn
log10/9 n
1
81/1000 n
729/1000 n
cn
≤ cn
1
≤ cn
O(n lg n)

平衡的划分

- 树的每一层都有代价cn，直到在深度$\log_{10}n=\Theta(\lg n)$处达到边界条件，以后各层代价至多为cn。
- 递归于深度$\log_{10/9}n=\Theta(\lg n)$处结束。
- 快速排序的总时间代价为$T(n)=\Theta(n\log n)$，从渐进意义上看就和划分是在中间进行的一样。

平衡的划分

- 事实上，即使是99:1的划分时间代价也为$\Theta(n\log n)$。
- 任何一种按常数比例进行划分所产生的递归树的深度都为$\Theta(\lg n)$，其中每一层的代价为$O(n)$，因而不管常数比例是什么，总的运行时间都为$\Theta(n\log n)$，只不过其中隐含的常数因子有所不同。

平衡情况性能的直觉分析

- 当我们对一个随机的输入数组应用快速排序时，要想在每一层上都有同样的划分是不太可能的。我们所能期望的是某些划分较对称，另一些则很不对称。
- 事实上，我们可以证明，如果选择L[p..r]的第一个元素作为支点元素，Partition所产生的划分80%以上都比9:1更对称，而另20%则比9:1差。

平衡情况性能的直觉分析

- 平均情况下，Partition产生的划分中既有“好的”，又有“差的”。
- 与Partition执行过程对应的递归树中，好、差划分是随机地分布在树的各层上的。
- 为与我们的直觉相一致，假设好、差划分交替出现在树的各层上，且好的划分是最佳情况划分，而差的划分是最坏情况下的划分，

平衡情况性能的直觉分析

- 从直觉上看，当好、差划分交替分布划分都是好的一样：仍是Θ (nlogn)，但Θ记号中隐含的常数因子要略大一些。关于平均情况的严格分析将在后文给出。

7.3 快速排序的随机化版本

- 前面分析中，我们假定输入数据的所有排列都是等可能的。但在实际应用中，这个假设就不会总是成立。
- 解决的方法是，利用随机化策略，能够克服分布的等可能性假设所带来的问题。

7.3 快速排序的随机化版本

- 快速排序的随机化版本有一个和其他随机化算法一样的有趣性质：
  - 没有一个特别的输入会导致最坏情况性态。
- 随机算法的最坏情况性态是由随机数产生器决定的。
  - 即使有意给出一个坏的输入也没用，因为随机化排列会使得输入数据的次序对算法不产生影响。
  - 只有在随机数产生器给出了一个很不巧的排列时，随机化算法的最坏情况性态才会出现。
  - 事实上可以证明几乎所有的排列都可使快速排序接近平均情况性态,只有非常少的几个排列才会导致算法的近最坏情况性态。

7.3 快速排序的随机化版本

- 我们在前文已经了解到，对快速排序来说，一组好坏相杂的划分仍能产生很好的运行时间。因此我们可以认为该算法的随机化版本也能具有较好的性能。
- 我们可以向算法中加入随机化的成分，以便对于所有输入，均能获得较好的平均情况性能。
- 在这里，我们采用随机采样的随机化技术，从子数组A[p..r]中随机选择一个元素，即将A[r]与从A[p..r]中随机选出的一个元素交换。
- 在这一修改中，我们是从p,…,r这一范围中随机取样的，确保了在子数组的r-p+1个元素中，主元元素x=A[r]等可能地取其中的任何一个。

随机化快排

- RANDOMIZED-PARTITION(A,p,r)

  1 i←RANDOM(p,r)

  2 exchange A[r]←→A[i]

  3 return PARTITION(A,p,r)

- 新的快速排序过程调用RANDOMIZED-PARTITION

  RANDOMIZED-PARTITION(A,p,r)

  1 if p < r

  2 then q←RANDOMIZED-PARTITION(A,p,r)

  3 RANDOMIZED-PARTITION(A,p,q-1)

  4 RANDOMIZED-PARTITION(A,q+1,r)

7.4 快速排序分析和最坏情况分析

- 快速排序分析
- 在前文我们从直觉上分析了快速排序在平均情况下的性能为Θ (nlogn)，我们将在下面定量地分析快速排序法的性能。

- 最坏情况分析:
- 前面从直觉上可以判断出最坏情况发生在每次划分过程产生的两个区间分别包含n-1个元素和1个元素的时候(设输入的表有n个元素)。下面我们来证明这种每次划分过程产生的两个区间分别包含n-1个元素和1个元素的情况就是最坏情况

7.4 最坏情况分析

- 设T(n)是过程Quick_Sort作用于规模为n的输入上的最坏情况的时间，则

$$T(n)=\max_{0\le q\le n-1}(T(q)+T(n-q-1)+\Theta(n)) \quad (7.1)$$

其中1≤q≤n-1，因为PARTITION过程产生两个区域，每个大小至多为n-1。我们假设$T(n)\le cn^2$，其中c为常数。将归纳假设代入上式，得到：

$$T(n)\le \max_{0\le q\le n-1}(cq^2+c(n-q-1)^2+\Theta(n))$$

$$=c\max_{0\le q\le n-1}(q^2+(n-q-1)^2+\Theta(n))$$

- 因为在[0,n-1]上$q^2+(n-q-1)^2$关于q递减，所以当q=0时$q^2+(n-q-1)^2$有最大值n^2-2n+1。

7.4 最坏情况分析

- 于是有：

$$T(n) \le cn^2 - 2c(2n-1) + \Theta(n))$$

$$= cn^2$$

- 只要c足够大，上面的第二个小于等于号就可以成立。这样快速排序最坏情况为$\Theta(n^2)$

7.4 平均情况分析

- QUICKSORT的运行时间是由花在过程PARTITION上的时间所决定的。
- 每当PARTITION过程被调用时，就要选出一个主元素。于是最多调用这个过程n次
- 调用一次PARTITION件事与3~6行for循环次数成正比，这个for循环每一轮迭代都要在第4行中进行一次比较
- 只要知道第4行执行次数，就可以知道QUICKSORT中for循环所化时间

7.4 平均情况分析

- 引理7.1 设当QUICKSORT在一个包含n个元素的数组上运行时，PARTITION在第四行所做比较次数为X。那么，QUICKSORT的运行时间为O(n+X)
- 证明：根据上面的讨论，对PARTITION的调用共有n次，每一次调用都需做固定量的工作，再执行若干次for循环。在for循环的每一轮迭代中，都需要执行第4行。

7.4 快速排序分析

- 我们定义指示器随机变量

$$X_{ij} = I\{z_i \text{与} z_j \text{进行比较}\}$$

由于每个元素只和主元元素进行比较，而且之后该主元元素不会再跟其他元素比较了，因此总的比较次数

$$X = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} X_{ij}$$

对上式求期望值，得到

$$\mathrm{E}[X] = E\left[\sum_{i=1}^{n-1} \sum_{j=i+1}^{n} X_{ij}\right] = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} E\left[X_{ij}\right] = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} \Pr\{z_i \text{与} z_j \text{进行比较}\}$$

7.4 平均情况分析

- 在 z_{ij} 中的某一元素被选为主元之前，集合 z_{ij} 整个都是在统一划分中的。于是，z_{ij} 中的任何元素都会等可能地被首先选为主元。因为集合 z_{ij} 中共有j-i+1个元素，还因为主元是随即独立选取的，所以，任何元素被首先选为主元的概率是1/(j-i+1)于是

$$P_r\{z_i\text{与}z_j\text{进行比较}\} = P_r\{z_i\text{或是}z_j\text{是从}Z_{ij}\text{中首先选出的主元}\}$$

$$= P_r\{z_i\text{是从}Z_{ij}\text{中首先选出的主元}\}$$

$$+ P_r\{z_j\text{是从}Z_{ij}\text{中首先选出的主元}\}$$

$$= \frac{1}{j-i+1} + \frac{1}{j-i+1} = \frac{2}{j-i+1}$$

7.4 平均情况分析

- 综合起来，有

$$E[X]=\sum_{i=1}^{n-1}\sum_{j=i+1}^{n}\frac{2}{j-i+1}$$

另k=j-i，有

$$E[X]=\sum_{i=1}^{n-1}\sum_{j=i+1}^{n}\frac{2}{j-i+1}=\sum_{i=1}^{n-1}\sum_{k=1}^{n-i}\frac{2}{k+1}<\sum_{i=1}^{n-1}\sum_{k=1}^{n}\frac{2}{k}=\sum_{i=1}^{n-1}O(\lg n)=O(n\lg n)$$

课堂练习7.1

设A是n个非o实数构成的数组，设计一个算法重新排列数组中的数，使得负数都排在正数前面。要求算法使用O(n)的时间和O(1)的空间。